エンジニア研修 ・ 2026年度版

Optional

「値が無いかもしれない」を、戻り値の型で伝える

map・filter　orElse　null 安全

© 2026 by yo1000 | Yoichi KIKUCHI is licensed under CC BY-ND 4.0

この講義の流れ

Optional で「無いかもしれない」を扱う

	テーマ	内容
01	Optional を使う理由	「無いかもしれない」を型で表す
02	Optional のメソッド	map・filter と null 安全のためのメソッド
03	Optional の組み合わせ演習	Stream と組み合わせて使う
	まとめ	やりたいこと別 早見表

Stream API の講義の続きです。Stream の map・filter・flatMap や findFirst を使います。サンプルは JShell で実行し、出力を自分の目で確かめながら進めます。

はじめに

演習準備：Docker経由でJShellを使う

サンプルの実行と演習では、Docker経由でJShellを使用します。以下のコマンドでJShellを実行できるようにしておきます。

```
docker container run --name edu2026_stream -ti --rm amazoncorretto:25 jshell
```

JShellとは

Java Shellの略で、JavaのREPL環境としてJava9以降に導入された対話型ツールです。

REPLとは

Read-Eval-Print Loopの略で、入力内容をその場で読み込み（Read）評価し（Eval）表示する（Print）を繰り返す実行環境のことで、小さなプログラムの実行確認などに使用されるものです。

はじめに

複数行のコードは { } で囲んで貼り付ける

JShell は、1行ごとに「ここで文が終わっているか」を判断して実行します。行頭が「.」で始まるつなぎ方は、1行目の時点で文が終わったと判断されてしまいます。このスライド以降のサンプルは、すべて { } で囲んだ形で載せているので、そのままコピーして貼り付けてください。

そのまま貼り付けると

```
List<String> names = trainees.stream()
    .map(Trainee::name)
    .toList();
```

JShell の出力

```
|  Error:
|  incompatible types:
java.util.stream.Stream<Trainee> cannot be converted
to java.util.List<java.lang.String>
（以下略。2行目・3行目もエラーになる）
```

{ } で囲んで貼り付けると

```
{
    List<String> names = trainees.stream()
        .map(Trainee::name)
        .toList();
    System.out.println(names);
}
```

JShell の出力

```
[佐藤, 鈴木, 高橋, 田中, 伊藤, 渡辺]
```

はじめに

サンプルデータを JShell に読み込む

最初に、次のコードを JShell に貼り付けておきます。JShell を終了したら、もう一度貼り付けます。

```
record Trainee(String name, String team, int score) {}

var trainees = List.of(
    new Trainee("佐藤", "A", 82),
    new Trainee("鈴木", "B", 65),
    new Trainee("高橋", "A", 91),
    new Trainee("田中", "C", 48),
    new Trainee("伊藤", "B", 77),
    new Trainee("渡辺", "C", 70));
```

読み方

- record は、値を持つだけのクラスを短く書ける構文
- t.name()、t.team()、t.score() で各値を取り出せる
- 以降のサンプルは、すべてこの trainees を使う

01

Optionalを使う理由

「値が無いかもしれない」を、戻り値の型で伝える

Optional を使う理由

null かもしれないことは、型からは分からない

```
{
    Trainee found = null;
    for (Trainee t : trainees) {
        if (t.name().equals("山田")) {
            found = t;
        }
    }
    System.out.println(found.score()); // 山田さんはいない
}
```

実行結果

```
|  Exception java.lang.NullPointerException: Cannot invoke "…Trainee.score()" because "<local0>" is null
|        …（スタックトレースは省略）
```

ポイント

- for 文で探して見つからないと、found は null のまま。型は Trainee なので、null かもしれないことが型から分からない
- null チェックを書き忘れても、コンパイルは通ってしまう。問題が分かるのは実行したとき

Optional を使う理由

Optional は「無いかもしれない」を戻り値の型で表す

```
{
    Optional<Trainee> found = trainees.stream()
        .filter(t -> t.name().equals("山田"))
        .findFirst();
    System.out.println(found);
}
```

実行結果

```
Optional.empty
```

ポイント

- Stream の findFirst の戻り値の型は Optional<Trainee>。型を見ただけで「見つからない場合がある」と分かる
- 見つからなければ null ではなく、空の Optional（Optional.empty）が返る
- 中身を取り出すには「無かったときの扱い」を決めるメソッドを呼ぶので、null チェックの書き忘れが起きない

Optional を使う理由

Optional は「0個か1個の値」を入れる箱

高橋さんを探した場合と、山田さんを探した場合の findFirst の結果です。

値がある（高橋さん）

Optional[Trainee[name=高橋, team=A, score=91]]
中に値が1つ入っている

値がない（山田さん）

Optional.empty
中身が空

null の代わりに「空の箱」を返します。箱のまま map や filter で加工し、最後に取り出します。

Optional を使う理由

Stream と Optional は、似たメソッドを持つが目的が違う

	Stream	Optional
目的	複数の要素を、まとめて加工・集計する	値が無いかもしれないことを伝え、無いときの扱いを決めさせる
中身の数	0〜n 個	0 個か 1 個
主な使い場所	コレクションの加工・集計	メソッドの戻り値
実行のタイミング	終端操作を呼ぶまで実行されない	map や filter を呼んだその場で実行される
共通するメソッド	map・filter・flatMap	map・filter・flatMap（中身があるときだけ適用する）

Optional を使う理由

Optional の map は、呼んだその場で実行される

```
{
    Optional<String> o = Optional.of("高橋")
        .map(n -> {
            System.out.println("map 実行");
            return n + "さん";
        });
    System.out.println("--- 終端操作は無い ---");
    System.out.println(o);
}
```

実行結果

```
map 実行
--- 終端操作は無い ---
Optional[高橋さん]
```

ポイント

- Optional には終端操作が無い。map を呼んだ時点で中身に適用される
- Stream のような「流れ」ではなく、値を入れた箱だと考える

Optional を使う理由

値が無いかもしれないことを、どう表すか

場面	書き方	理由
メソッドの戻り値で、値を返せないことがある（探す・最初の1件・最大値）	Optional<T> を返す	型を見ただけで「無い場合がある」と分かり、呼び出し側が扱いを決められる
メソッドの戻り値が、必ずある	T をそのまま返す（値が無い場合は例外を投げる）	値が無いのは異常な状態なので、null や Optional ではなく例外で知らせる
メソッドの戻り値が、複数件（0件もある）	空の List を返す	List は「0件」をそれ自体で表せる
フィールド・引数・ローカル変数	T のまま使う（null の設定があり得る）	Optional は戻り値のための型。null を許容しない場合は、受け取った所で判定する
null を返す既存のメソッド（Map の get など）	Optional.ofNullable で包む	包めば、以降は Optional のメソッドで安全に扱える

迷ったら「このメソッドは、値を返せないことがあるか？」で判断します。あるなら Optional、無いなら普通の型を返します。

Optional を使う理由

Optional を使うときの約束

Optional の変数や戻り値に、null を入れない

無いときは Optional.empty() を返す。Optional 自体が null だと、受け取った側で NullPointerException になり、Optional にした意味がなくなる

get() でいきなり取り出さない

空のときは例外になり、null チェックを忘れたのと同じことが起きる。orElse・orElseThrow など、無いときの扱いを決めるメソッドで取り出す

すぐに取り出さず、箱のまま加工する

map・filter で箱のまま加工し、最後に1回だけ取り出す。isPresent と get の組み合わせは、null チェックと同じ書き方に戻ってしまう

この約束を守れば、Optional を受け取った側は null を気にせずに書けます。

02

Optional のメソッド

Stream と似たメソッドと、null 安全のためのメソッド

Optional のメソッドの全体像

戻り値が Optional か、それ以外かで2つに分かれる

Optional を返す → さらにつなげられる

メソッド	役割
map	中身を変換する
filter	条件に合わなければ空にする
flatMap	Optional を返す変換をつなぐ
or	空なら別の Optional で補う

Optional 以外を返す → 取り出す・処理する

メソッド	役割
orElse / orElseGet	空なら代わりの値
orElseThrow	空なら例外を投げる
ifPresent	有るときに処理する
ifPresentOrElse	有る・無いで処理を分ける
isPresent / isEmpty	有無を boolean で返す
stream	0件か1件の Stream にする

map・filter・flatMap は Stream と同じ形で、中身があるときだけ適用されます。作るときは Optional.of / ofNullable / empty を使います。

Optional ・ 作る

of / ofNullable / empty：Optional を作る（1/2）

値から Optional を作ります。null かもしれない値を包むときは ofNullable を使います。

`Optional.of(値) ／ Optional.ofNullable(値) ／ Optional.empty()`

```
{
    Optional<Trainee> a = Optional.of(trainees.get(0));
    Optional<Trainee> b = Optional.empty();
    System.out.println(a);
    System.out.println(b);
}
```

実行結果

```
Optional[Trainee[name=佐藤, team=A, score=82]]
Optional.empty
```

of / ofNullable / empty：Optional を作る（2/2）

```
{
    Trainee found = null; // for 文で探して、見つからなかったと仮定
    Optional<Trainee> c = Optional.ofNullable(found);
    System.out.println(c);
    Optional<Trainee> d = Optional.of(found);
}
```

実行結果

```
Optional.empty
|  Exception java.lang.NullPointerException
|        …（スタックトレースは省略）
```

ポイント

・of：null でない値に使う。null なら即例外

・ofNullable：null なら空になる

・empty：「無い」ことを返すときに使う

演習 ・ Optional ・ 作る

for 文で探した結果を Optional で包む

次のコードは for 文で山田さんを探します。見つからなければ found は null のままです。found を Optional で包んで Optional<Trainee> 型の変数に入れ、その変数を出力するように書き換えてください。「Optional.empty」と出力されれば完成です。

ヒント：null かもしれない値を包むときは Optional.ofNullable

trainees　佐藤 A 82 ／ 鈴木 B 65 ／ 高橋 A 91 ／ 田中 C 48 ／ 伊藤 B 77 ／ 渡辺 C 70

```
{
    Trainee found = null;
    for (Trainee t : trainees) {
        if (t.name().equals("山田")) {
            found = t;
        }
    }
    System.out.println(found);
}
```

JShell に貼り付けて実行し、結果を確かめます。複数行のコードは、全体を { } で囲んで貼り付けます。

Optional ・ Streamと似たメソッド ・ map

map：中身があれば変換する（1/2）

値があれば変換し、結果を Optional に入れ直して返します。空なら何もせず、空のまま返します。

```
Optional<U> map(Function<T, U> mapper)
```

```
{
    Optional<Trainee> takahashi = trainees.stream()
        .filter(t -> t.name().equals("高橋"))
        .findFirst();
    Optional<String> team = takahashi.map(t -> t.team());
    System.out.println(team);
}
```

実行結果

```
Optional[A]
```

map：中身があれば変換する（2/2）

```
{
    Optional<Trainee> yamada = trainees.stream()
        .filter(t -> t.name().equals("山田"))
        .findFirst(); // Optional.empty
    Optional<String> none = yamada.map(t -> t.team()); // 実行されない
    System.out.println(none);
}
```

実行結果

```
Optional.empty
```

ポイント

・findFirst の結果を Optional の変数に入れてから map を呼ぶと、Optional の map だと分かりやすい

・空のときはラムダ式が呼ばれないので、null チェックがいらない

・Stream の map と同じ形で使える

演習 ・ Optional ・ Streamと似たメソッド ・ map

点数を「77点」の文字列にする

foundの点数を「77点」の形式の文字列に変換して出力してください。

ヒント：found に map をつなぐ

trainees　佐藤 A 82 ／ 鈴木 B 65 ／ 高橋 A 91 ／ 田中 C 48 ／ 伊藤 B 77 ／ 渡辺 C 70

```
{
    for (String name : List.of("伊藤", "山田")) {
        Optional<Trainee> found = trainees.stream()
            .filter(t -> t.name().equals(name)).findFirst();
        System.out.println(found); // ここを書き換える
    }
}
```

期待する結果

```
Optional[77点]
Optional.empty
```

JShell に貼り付けて実行し、結果を確かめます。複数行のコードは、全体を { } で囲んで貼り付けます。

Optional ・ Streamと似たメソッド ・ filter

filter：条件に合わなければ空にする（1/2）

値があり、かつ条件を満たすときだけ値を残します。満たさなければ空の Optional を返します。

```
Optional<T> filter(Predicate<T> predicate)
```

```
{
    Optional<Trainee> sato = trainees.stream()
        .filter(t -> t.name().equals("佐藤"))
        .findFirst();
    Optional<Trainee> a = sato.filter(t -> t.score() >= 70); // 82点なので残る
    System.out.println(a);
}
```

実行結果

```
Optional[Trainee[name=佐藤, team=A, score=82]]
```

filter：条件に合わなければ空にする（2/2）

```
{
    Optional<Trainee> suzuki = trainees.stream()
        .filter(t -> t.name().equals("鈴木"))
        .findFirst();
    Optional<Trainee> b = suzuki.filter(t -> t.score() >= 70); // 65点なので空になる
    System.out.println(b);
}
```

実行結果

```
Optional.empty
```

ポイント

- 「見つからない」と「条件に合わない」を、同じ「空」として扱える
- Stream の filter と同じ形

演習 ・ Optional ・ Streamと似たメソッド ・ filter

Aチームの研修生だけ名前を取り出す

foundがAチームの研修生のときだけ、名前を取り出して出力してください。

ヒント：found を filter で絞り込み、map で名前にする

trainees　佐藤 A 82 ／ 鈴木 B 65 ／ 高橋 A 91 ／ 田中 C 48 ／ 伊藤 B 77 ／ 渡辺 C 70

```
{
    for (String name : List.of("高橋", "鈴木")) {
        Optional<Trainee> found = trainees.stream()
            .filter(t -> t.name().equals(name)).findFirst();
        System.out.println(found); // ここを書き換える
    }
}
```

期待する結果

```
Optional[高橋]
Optional.empty
```

JShell に貼り付けて実行し、結果を確かめます。複数行のコードは、全体を { } で囲んで貼り付けます。

Optional ・ Streamと似たメソッド ・ flatMap

flatMap：Optional を返す変換を平らにつなぐ（1/3）

変換の結果が Optional のとき、二重に包まずにそのまま返します。

```
Optional<U> flatMap(Function<T, Optional<U>> mapper)
```

```
{
    Optional<Trainee> sato = trainees.stream()
        .filter(t -> t.name().equals("佐藤"))
        .findFirst();
    Optional<Optional<Trainee>> nested = sato
        .map(me -> trainees.stream()
            .filter(o -> o.team().equals(me.team()) && o.score() > me.score())
            .findFirst());
    System.out.println(nested);
}
```

実行結果

```
Optional[Optional[Trainee[name=高橋, team=A, score=91]]]
```

flatMap：Optional を返す変換を平らにつなぐ（2/3）

```
{
    Optional<Trainee> sato = trainees.stream()
        .filter(t -> t.name().equals("佐藤"))
        .findFirst();
    Optional<Trainee> higher = sato
        .flatMap(me -> trainees.stream()
            .filter(o -> o.team().equals(me.team()) && o.score() > me.score())
            .findFirst());
    System.out.println(higher);
}
```

実行結果

```
Optional[Trainee[name=高橋, team=A, score=91]]
```

flatMap：Optional を返す変換を平らにつなぐ（3/3）

```
{
    Optional<Trainee> takahashi = trainees.stream()
        .filter(t -> t.name().equals("高橋"))
        .findFirst();
    Optional<Trainee> none = takahashi
        .flatMap(me -> trainees.stream()
            .filter(o -> o.team().equals(me.team()) && o.score() > me.score())
            .findFirst());
    System.out.println(none);
}
```

実行結果

```
Optional.empty
```

ポイント

- 例：佐藤さんを探し、同じチームで佐藤さんより点数が高い研修生を探す
- map だと Optional<Optional<Trainee>> になってしまう
- 「探す → その結果でさらに探す」をつなげられる

演習 ・ Optional ・ Streamと似たメソッド ・ flatMap

同じチームで点数が高い研修生を探す

found と同じチームで、found より点数が高い研修生の名前を出力してください。

ヒント：サンプルの flatMap を found につなぎ、最後に map で名前にする

trainees　佐藤 A 82 ／ 鈴木 B 65 ／ 高橋 A 91 ／ 田中 C 48 ／ 伊藤 B 77 ／ 渡辺 C 70

```
{
    for (String name : List.of("田中", "渡辺")) {
        Optional<Trainee> found = trainees.stream()
            .filter(t -> t.name().equals(name)).findFirst();
        System.out.println(found); // ここを書き換える
    }
}
```

期待する結果

```
Optional[渡辺]
Optional.empty
```

JShell に貼り付けて実行し、結果を確かめます。複数行のコードは、全体を { } で囲んで貼り付けます。

Optional ・ null安全 ・ or

or：空なら別の Optional で補う

空のときだけ、代わりの Optional を作る処理を実行します。戻り値も Optional です（Java 9以降）。

```
Optional<T> or(Supplier<Optional<T>> supplier)
```

```
{
    Optional<Trainee> over95 = trainees.stream()
        .filter(t -> t.score() >= 95)
        .findFirst(); // 95点以上はいない
    Optional<Trainee> found = over95
        .or(() -> trainees.stream()
            .filter(t -> t.score() >= 90)
            .findFirst()); // 90点以上で探し直す
    System.out.println(found);
}
```

実行結果

```
Optional[Trainee[name=高橋, team=A, score=91]]
```

Optional ・ null安全 ・ or

or：空なら別の Optional で補う（ポイント）

ポイント

・「まずこちらを探し、無ければあちらを探す」を順に書ける

・値があるときは、代わりの処理は実行されない

演習 ・ Optional ・ null安全 ・ or

Dチームがいなければ Cチームの研修生

Dチームの最初の研修生を探し、いなければ Cチームの最初の研修生を使うようにして、結果を出力してください。

trainees　佐藤 A 82 ／ 鈴木 B 65 ／ 高橋 A 91 ／ 田中 C 48 ／ 伊藤 B 77 ／ 渡辺 C 70

期待する結果

```
Optional[Trainee[name=田中, team=C, score=48]]
```

JShell に貼り付けて実行し、結果を確かめます。複数行のコードは、全体を { } で囲んで貼り付けます。

Optional ・ null安全 ・ orElse

orElse：空なら代わりの値を使う（1/2）

値があればその値を、空なら引数の値を返します。戻り値は Optional ではなく中身の型です。

`T orElse(T other)`

```
{
    Optional<Trainee> teamA = trainees.stream()
        .filter(t -> t.team().equals("A"))
        .findFirst();
    String a = teamA.map(t -> t.name()).orElse("該当なし");
    System.out.println(a);
}
```

実行結果

佐藤

orElse：空なら代わりの値を使う（2/2）

```
{
    Optional<Trainee> teamD = trainees.stream()
        .filter(t -> t.team().equals("D"))
        .findFirst();
    String d = teamD.map(t -> t.name()).orElse("該当なし");
    System.out.println(d);
}
```

実行結果

該当なし

ポイント

・最後に値を取り出すときに使う

・引数の値は、Optional が空でなくても必ず作られる（次の orElseGet で確かめる）

演習 ・ Optional ・ null安全 ・ orElse

見つからなければ0点として扱う

found の点数を出力してください。研修生が見つからないときは 0 を出力してください。

ヒント：map で点数に変換してから orElse

trainees　佐藤 A 82 ／ 鈴木 B 65 ／ 高橋 A 91 ／ 田中 C 48 ／ 伊藤 B 77 ／ 渡辺 C 70

```
{
    for (String name : List.of("高橋", "山田")) {
        Optional<Trainee> found = trainees.stream()
            .filter(t -> t.name().equals(name)).findFirst();
        System.out.println(found); // ここを書き換える
    }
}
```

期待する結果

```
91
0
```

JShell に貼り付けて実行し、結果を確かめます。複数行のコードは、全体を { } で囲んで貼り付けます。

Optional ・ null安全 ・ orElseGet

準備：呼ばれたことが分かるメソッドを用意する

orElse と orElseGet の違いを確かめるために、次のメソッドを JShell に貼り付けておきます。

```
String fallback() {
    System.out.println("fallback 実行");
    return "該当なし";
}
```

実行結果

```
|  created method fallback()
```

ポイント

- fallback() は「fallback 実行」と出力してから "該当なし" を返す
- orElse の引数は値（String）なので、ラムダ式は渡せない。呼ばれたことが出力で分かるメソッドにしておく

注目　「fallback 実行」が出力されたかどうかで、fallback() が呼ばれたかが分かります。

Optional ・ null安全 ・ orElseGet

orElseGet：空のときだけ代わりの値を作る（1/2）

空のときだけ、代わりの値を作る処理を実行します。

`T orElseGet(Supplier<T> supplier)`

```
{
    Optional<Trainee> teamA = trainees.stream()
        .filter(t -> t.team().equals("A"))
        .findFirst();
    String a = teamA.map(t -> t.name())
        .orElse(fallback()); // 値があっても fallback が呼ばれる
    System.out.println(a);
}
```

実行結果

```
fallback 実行
佐藤
```

注目　Aチームには佐藤さんがいる（値がある）のに、「fallback 実行」が出力されている

Optional ・ null安全 ・ orElseGet

orElseGet：空のときだけ代わりの値を作る（2/2）

```
{
    Optional<Trainee> teamA = trainees.stream()
        .filter(t -> t.team().equals("A"))
        .findFirst();
    String b = teamA.map(t -> t.name())
        .orElseGet(() -> fallback()); // 値があるので呼ばれない
    System.out.println(b);
}
```

実行結果

佐藤

注目　値があるので、「fallback 実行」は出力されない（ラムダ式が呼ばれていない）

ポイント

・orElse の引数は、値があっても実行される

・作るのが重い値（DB・API の呼び出しなど）は orElseGet、定数なら orElse で十分

演習 ・ Optional ・ null安全 ・ orElseGet

出力される行を予想する

fallback() は「fallback 実行」と出力してから "該当なし" を返すメソッドです（JShell に貼り付け済み）。次のコードを実行したとき、何がどの順番で出力されるかを予想してから、実行して確かめてください。

trainees　佐藤 A 82 ／ 鈴木 B 65 ／ 高橋 A 91 ／ 田中 C 48 ／ 伊藤 B 77 ／ 渡辺 C 70

```
{
    Optional<String> first = trainees.stream().map(t -> t.name()).findFirst();
    Optional<String> teamD = trainees.stream()
        .filter(t -> t.team().equals("D"))
        .map(t -> t.name())
        .findFirst();
    String x = teamD.orElseGet(() -> fallback());
    String y = first.orElseGet(() -> fallback());
    String z = first.orElse(fallback());
    System.out.println(x + " / " + y + " / " + z);
}
```

JShell に貼り付けて実行し、結果を確かめます。複数行のコードは、全体を { } で囲んで貼り付けます。

Optional ・ null安全 ・ orElseThrow

orElseThrow：空なら例外を投げる（1/2）

値があればその値を返し、空なら例外を投げます。「無いのはおかしい」場面で使います。

`T orElseThrow() ／ T orElseThrow(Supplier<X> exceptionSupplier)`

```
{
    Optional<Trainee> takahashi = trainees.stream()
        .filter(t -> t.name().equals("高橋"))
        .findFirst();
    Trainee t1 = takahashi.orElseThrow();
    System.out.println(t1.score());
}
```

実行結果

```
91
```

orElseThrow：空なら例外を投げる（2/2）

```
{
    Optional<Trainee> yamada = trainees.stream()
        .filter(t -> t.name().equals("山田"))
        .findFirst();
    Trainee t2 = yamada.orElseThrow(() -> new IllegalArgumentException(
        "研修生が見つかりません: 山田"));
}
```

実行結果

```
|  Exception java.lang.IllegalArgumentException: 研修生が見つかりません: 山田
|        …（スタックトレースは省略）
```

ポイント

- 引数なしは NoSuchElementException を投げる（Java 10以降）
- 例外の種類とメッセージを指定すると、原因が分かりやすい
- get() も同じ動きだが、例外を投げることが名前から分からないので使わない

演習 ・ Optional ・ null安全 ・ orElseThrow

研修生がいなければ例外にする

Dチームの最初の研修生を取り出してください。

いない場合は、メッセージ「Dチームの研修生がいません」の IllegalStateException を投げるようにしてください。

trainees　佐藤 A 82 ／ 鈴木 B 65 ／ 高橋 A 91 ／ 田中 C 48 ／ 伊藤 B 77 ／ 渡辺 C 70

期待する結果

```
|  Exception java.lang.IllegalStateException: Dチームの研修生がいません
|        …（スタックトレースは省略）
```

JShell に貼り付けて実行し、結果を確かめます。複数行のコードは、全体を { } で囲んで貼り付けます。

Optional ・ null安全 ・ ifPresent

ifPresent：値があるときだけ処理する（1/2）

値があるときだけ、引数の処理を実行します。戻り値はありません。

```
void ifPresent(Consumer<T> action)
```

```
{
    Optional<Trainee> ito = trainees.stream()
        .filter(t -> t.name().equals("伊藤"))
        .findFirst();
    ito.ifPresent(t -> System.out.println(t.name() + "さん：" + t.score() + "点"));
}
```

実行結果

伊藤さん：77点

Optional ・ null安全 ・ ifPresent

ifPresent：値があるときだけ処理する（2/2）

```
{
    Optional<Trainee> yamada = trainees.stream()
        .filter(t -> t.name().equals("山田"))
        .findFirst();
    yamada.ifPresent(t -> System.out.println(t.name() + "さん：" + t.score() + "点"));
}
```

実行結果

（何も出力されない）

ポイント

- if (x != null) { … } の置き換え
- 空のときは何も起きない

演習 ・ Optional ・ null安全 ・ ifPresent

優秀者がいれば紹介する

90点以上の研修生がいれば、最初の1人を「優秀者：高橋さん」の形式で出力してください。

trainees　佐藤 A 82 ／ 鈴木 B 65 ／ 高橋 A 91 ／ 田中 C 48 ／ 伊藤 B 77 ／ 渡辺 C 70

期待する結果

優秀者：高橋さん

JShell に貼り付けて実行し、結果を確かめます。複数行のコードは、全体を { } で囲んで貼り付けます。

Optional ・ null安全 ・ ifPresentOrElse

ifPresentOrElse：有るとき・無いときの処理を両方書く（1/2）

値があるときの処理と、空のときの処理を両方指定します（Java 9以降）。

```
void ifPresentOrElse(Consumer<T> action, Runnable emptyAction)
```

```
{
    Optional<Trainee> teamA = trainees.stream()
        .filter(t -> t.team().equals("A"))
        .findFirst();
    teamA.ifPresentOrElse(
        t -> System.out.println("Aチーム: " + t.name()),
        () -> System.out.println("Aチーム: 該当なし"));
}
```

実行結果

```
Aチーム: 佐藤
```

Optional ・ null安全 ・ ifPresentOrElse

ifPresentOrElse：有るとき・無いときの処理を両方書く（2/2）

```
{
    Optional<Trainee> teamD = trainees.stream()
        .filter(t -> t.team().equals("D"))
        .findFirst();
    teamD.ifPresentOrElse(
        t -> System.out.println("Dチーム: " + t.name()),
        () -> System.out.println("Dチーム: 該当なし"));
}
```

実行結果

```
Dチーム: 該当なし
```

ポイント

- if-else の両方をまとめて書ける
- 空のときの処理は、引数なしのラムダ式 () -> … で書く

演習 ・ Optional ・ null安全 ・ ifPresentOrElse

登録されていない研修生を知らせる

trainees から山田さんを探し、見つかれば「山田さん：◯点」、見つからなければ「山田さんは登録されていません」と出力してください。

trainees　佐藤 A 82 ／ 鈴木 B 65 ／ 高橋 A 91 ／ 田中 C 48 ／ 伊藤 B 77 ／ 渡辺 C 70

期待する結果

山田さんは登録されていません

JShell に貼り付けて実行し、結果を確かめます。複数行のコードは、全体を { } で囲んで貼り付けます。

Optional ・ null安全 ・ isPresent / isEmpty

isPresent / isEmpty：値の有無を調べる（1/2）

値があれば isPresent が true、空なら isEmpty が true を返します（isEmpty は Java 11以降）。値の有無そのものを boolean で知りたいときに使います。

`boolean isPresent() ／ boolean isEmpty()`

```
{
    Optional<Trainee> yamada = trainees.stream()
        .filter(t -> t.name().equals("山田"))
        .findFirst();
    System.out.println(yamada.isPresent());
    System.out.println(yamada.isEmpty());
}
```

実行結果

```
false
true
```

isPresent / isEmpty：値の有無を調べる（2/2）

```
{
    Optional<Trainee> found = trainees.stream()
        .filter(t -> t.name().equals("山田"))
        .findFirst();
    if (found.isPresent()) { // nullチェックと同じ形
        System.out.println(found.get().score());
    } else {
        System.out.println("登録なし");
    }
}
```

実行結果

登録なし

非推奨　isPresent で確かめてから get で取り出すと、null チェックと同じ形に戻ってしまう。チェックを忘れれば get で例外になる。値を取り出すときは orElse・map などを使う

演習 ・ Optional ・ null安全 ・ isPresent / isEmpty

isPresent と get を使わずに書き直す

次のコードと同じ結果（Bチーム1位: 伊藤）になるように、isPresent と get を使わずに書き直してください。

ヒント：map で「Bチーム1位: ◯◯」に変換し、orElse で空のときの文字列を指定する

trainees　佐藤 A 82 ／ 鈴木 B 65 ／ 高橋 A 91 ／ 田中 C 48 ／ 伊藤 B 77 ／ 渡辺 C 70

```
{
    Optional<Trainee> top = trainees.stream()
        .filter(t -> t.team().equals("B"))
        .max(Comparator.comparing(Trainee::score));
    String label;
    if (top.isPresent()) {
        label = "Bチーム1位: " + top.get().name();
    } else {
        label = "Bチーム1位: なし";
    }
    System.out.println(label);
}
```

JShell に貼り付けて実行し、結果を確かめます。複数行のコードは、全体を { } で囲んで貼り付けます。

Optional ・ stream

stream：0件か1件の Stream にする

値があれば1件、空なら0件の Stream にします（Java 9以降）。Stream の中で空の Optional を取り除くときに使います。

```
Stream<T> stream()
```

```
{
    List<String> names = Stream.of("A", "B", "C", "D")
        .map(team -> trainees.stream()
            .filter(t -> t.team().equals(team) && t.score() >= 75)
            .findFirst()) // Stream<Optional<Trainee>>
        .flatMap(o -> o.stream()) // Stream<Trainee>：空は消える
        .map(t -> t.name())
        .toList();
    System.out.println(names);
}
```

実行結果

```
[佐藤, 伊藤]
```

Optional ・ stream

stream：0件か1件の Stream にする（ポイント）

ポイント

・Stream の中に Optional が並んだとき、flatMap(o -> o.stream()) で空を捨てて中身だけを残せる

・filter(o -> o.isPresent()).map(o -> o.get()) と書かなくてよい

演習 ・ Optional ・ stream

見つかった研修生の点数だけを集める

Stream.of("佐藤", "山田", "田中", "木村") の各名前で研修生を探し、見つかった研修生の点数だけをリストにして出力してください。

ヒント：flatMap の中で名前で探し、findFirst の結果に .stream() をつなぐ

trainees　佐藤 A 82 ／ 鈴木 B 65 ／ 高橋 A 91 ／ 田中 C 48 ／ 伊藤 B 77 ／ 渡辺 C 70

期待する結果

```
[82, 48]
```

JShell に貼り付けて実行し、結果を確かめます。複数行のコードは、全体を { } で囲んで貼り付けます。

03

Optional の組み合わせ演習

Stream と Optional を組み合わせて使う

演習 ・ Optional 組み合わせ演習 1

研修生を紹介する

研修生の名前を受け取り、「佐藤：82点」の形式の文字列を返すメソッド describe を作ってください。

研修生が見つからないときは「山田：登録なし」の形式の文字列を返します。

describe("佐藤")・describe("山田") の結果を出力してください。

trainees　佐藤 A 82 ／ 鈴木 B 65 ／ 高橋 A 91 ／ 田中 C 48 ／ 伊藤 B 77 ／ 渡辺 C 70

期待する結果

```
|  created method describe(String)
佐藤：82点
山田：登録なし
```

JShell に貼り付けて実行し、結果を確かめます。複数行のコードは、全体を { } で囲んで貼り付けます。

演習 ・ Optional 組み合わせ演習 2

合否を判定する

研修生の名前を受け取り、70点以上なら「佐藤：合格」、70点未満なら「鈴木：再テスト」の形式の文字列を返すメソッド judge を作ってください。

研修生が見つからないときは、メッセージ「未登録: 山田」の IllegalArgumentException を投げます。

judge("佐藤")・judge("鈴木")・judge("山田") を順に出力してください。

trainees　佐藤 A 82 ／ 鈴木 B 65 ／ 高橋 A 91 ／ 田中 C 48 ／ 伊藤 B 77 ／ 渡辺 C 70

期待する結果

```
|  created method judge(String)
佐藤：合格
鈴木：再テスト
|  Exception java.lang.IllegalArgumentException: 未登録: 山田
|        …（スタックトレースは省略）
```

JShell に貼り付けて実行し、結果を確かめます。複数行のコードは、全体を { } で囲んで貼り付けます。

演習 ・ Optional 組み合わせ演習 3

チームごとの1位

A・B・C・D の各チームについて、最高点の研修生の名前を「A: 高橋」の形式にし、リストにして出力してください。

研修生がいないチームは「D: 該当なし」とします。

ヒント：チームごとに filter → max で Optional を作り、map と orElse で文字列にする

trainees　佐藤 A 82 ／ 鈴木 B 65 ／ 高橋 A 91 ／ 田中 C 48 ／ 伊藤 B 77 ／ 渡辺 C 70

期待する結果

```
[A: 高橋, B: 伊藤, C: 渡辺, D: 該当なし]
```

JShell に貼り付けて実行し、結果を確かめます。複数行のコードは、全体を {} で囲んで貼り付けます。

演習 ・ Optional 組み合わせ演習 4

1位と同じチームの研修生

最も点数が高い研修生を求め、その研修生と同じチームで、その研修生より点数が低い研修生の名前を出力してください。

該当する研修生がいなければ「なし」と出力します。

ヒント：max の結果は Optional なので、そのまま flatMap をつなげられる

trainees　佐藤 A 82 ／ 鈴木 B 65 ／ 高橋 A 91 ／ 田中 C 48 ／ 伊藤 B 77 ／ 渡辺 C 70

期待する結果

佐藤

JShell に貼り付けて実行し、結果を確かめます。複数行のコードは、全体を { } で囲んで貼り付けます。

まとめ

Optional：「無いかもしれない」を型で伝え、扱いを決める

戻り値で伝える

見つからないことがあるメソッドは Optional を返す。フィールド・引数・リストには使わない。

中身はつないで加工

map・filter・flatMap で、中身があるときだけ加工する。null チェックを書かなくてよい。

最後に取り出す

orElse・orElseGet・orElseThrow・ifPresentOrElse で、無いときの扱いを決めて取り出す。get は使わない。

Stream と Optional は似たメソッドを持ちますが、目的は別物です。Stream は「複数をまとめて処理する」、Optional は「無いかもしれないを伝える」。

まとめ

やりたいこと別 早見表

やりたいこと	書き方	戻り値
null かもしれない値を包む	Optional.ofNullable(値)	Optional<T>
null でない値を包む	Optional.of(値)	Optional<T>
「無い」を返す	Optional.empty()	Optional<T>
中身を変換する	map(x -> 値)	Optional<R>
条件に合わなければ空にする	filter(x -> 条件)	Optional<T>
Optional を返す処理をつなぐ	flatMap(x -> Optional)	Optional<R>
空なら別の Optional で補う	or(() -> Optional)	Optional<T>
空なら代わりの値	orElse(値)	T
空のときだけ代わりの値を作る	orElseGet(() -> 値)	T
空なら例外を投げる	orElseThrow(() -> 例外)	T
有るときだけ処理する	ifPresent(x -> 処理)	void
有る・無いで処理を分ける	ifPresentOrElse(x -> …, () -> …)	void
有無を調べる	isPresent() / isEmpty()	boolean
Stream の中で空を捨てる	flatMap(Optional::stream)	Stream<T>